Guidance for CW2 (Individual Work)

High Card Series Application

Assessment Weight: 60% of module

CW2 Brief

- **Objective:** Develop a complete Java application for the **High Card Series** game.
- **Game Overview:**
  - 2 to 5 players compete over **5 to 10 rounds**
  - Each round:
    - Every player receives **one card**
    - Highest card wins → **+3 points**
    - Tie → each gets **+1 point**
  - Non-winning cards are returned to the deck
- **End of Game Features:**
  - Optional card replacement (once per player)
  - Bonus points based on:
    - Longest consecutive card sequence
    - Highest number of same-suit cards
- **Final Outcome:**
  - Total score = round points + bonus points
  - Player with highest score wins (or draw if equal)

Deliveries Summary Table

Delivery	Component	Description
Delivery 1	High Card Series Application	Java implementation of the full game (zipped project + code PDFs)
Delivery 2	Design & Development Document	Explanation of design decisions, data structures, and algorithms
Delivery 3	Testing Document	Description of testing approach, test classes, and validation
Delivery 4	Demonstration Video	≤ 5 minutes video showing application + code walkthrough

- **Key Note:**
  - **Application code is compulsory** – without it, submission **cannot be marked**
  - **Demonstration video is compulsory** – without it, submission **cannot be marked**

Delivery 1 – High Card Series Application

- **What to Submit:**
  - A **ZIP file** of your Java project (**Export Project to Zip File**)
  - Optional: Each **Java file also exported as PDF** and integrated in the Project (this is to secure another copy of your code)

- **Core Requirements:**
  - Fully working implementation of the game
  - The application must be **playable in IntelliJ and logically correct**
  - Covers functionality levels (L1 → L9)
  - **Junit Tests:**
    - Multiple Junit Classes.
    - Each class include multiple Junit tests for a function/functions.
    - Comprehensive **JUnit test coverage**

- **Key Note:**
  - **Application code is compulsory** – without it, submission **cannot be marked**

Delivery 2 – Design and Development Document

- **What to Submit:**
  - A **PDF document** explaining your design and development process.

- **What to Include:**
  - System design (classes, relationships, structure)
  - Use of **data structures** (e.g., arrays, lists, collections)
  - Design must show clear understanding of **ADT (Abstract Data Types)**
  - Algorithms used in the game logic
  - Justification of design decisions
  - Discussion of **alternative approaches** (and why not chosen)

Delivery 3 – Testing Document

- **What to Submit:**
  - A **PDF document** describing your testing approach

- **What to Include:**
  - Well-documented and organised testing process
  - Description of your **testing strategy** (Scenarios)
  - Structured and systematic testing using **JUnit**
  - Evidence of testing different scenarios including normal cases and edge cases (e.g., ties, min/max players, rounds)

Delivery 4 – Demonstration Video

- **What to Submit:**
  - A **video (~ 5 minutes)** demonstrating your application

- **What to Show:**
  - **Working application demo** - Game running (rounds, scoring, results)
  - Key parts of your code
  - Voiceover explanation (mandatory)

- **Key Note:**
  - **Video is compulsory** – without it, submission **cannot be marked**

Submission Assessment Grid (Criterion vs Deliveries)

Criterion	Marks	Related Deliveries
Design & Planning	15	Delivery 2 (Design Document)
Implementation	15	Delivery 1 (Application)
Functionality (L1–L9)	40	Delivery 1 (Application)
Testing	20	Delivery 1 (Application) Delivery 3 (Testing Document + JUnit)
Video	10	Delivery 4 (Demonstration Video)

- **Key Insights:**
  - The **Application (Delivery 1)** carries the highest weight (Functionality + Implementation + Testing)
  - **Design (Delivery 2)** focuses on ADT and decision-making
  - **Testing (Delivery 3)** must include **JUnit evidence**
  - **Video (Delivery 4)** demonstrates understanding and ownership

What is ADT in Design and Planning

- **ADT = Abstract Data Type -** A way to define **what a data structure does**, not how it is implemented
- **In Design & Planning:**
  - Focus on:
    - **Operations (methods)** the system must support
    - Behaviour of objects (e.g., Player, Deck, Card)
- **Examples in This Coursework:**
  - **Deck ADT:** shuffle(), dealCard(), returnCard()
  - **Player ADT:** addCard(), calculateScore(), getHand()
- **Why It Matters:**
  - Encourages **clear system design**
  - Separates **what the system should do** from how it is coded
  - Leads to better structure and easier implementation

- **Link to Marking:** Strong ADT design = better marks in **Design & Planning (15%)**

What is ADT in Implementation

- **In Implementation:**
  - Define **classes and methods** based on ADT design
  - Decide:
    - Data structures (e.g., ArrayList, arrays)
    - Internal logic of methods
- **Example (Deck ADT):**
  - Design: `dealCard()`
  - Implementation:

```
public Card dealCard() {
    return deck.remove(0);
}
```

- **Key Principles:**
  - **Encapsulation** → hide internal data
  - **Clear interfaces** → methods reflect ADT operations
  - **Consistency** → matches design decisions

- **Link to Marking:** Directly impacts **Implementation marks (15%)**

What is JUnit in Testing

- **JUnit is** A framework used to write and run **automated tests** for Java programs
- **In This Coursework (Compulsory):**
  - You **must use JUnit** to test your application
  - Provide **JUnit test classes** in your submission
- **What JUnit Does:**
  - Tests individual methods (unit testing)
  - Checks if expected results match actual results
  - Helps detect errors early

- **Key Message:** No JUnit tests → **low marks in Testing**
- **Link to Marking:** Required for **Testing marks (20%)**

THE
END